

Reading Data

Data Analysis & Information Extraction

Table of contents

Data Wrangling	3
Basic Concepts	4
Understanding the path and working directory	4
Creating a path	4
Importing data	6
Reading Data	7
Downloading data	7
CSV files	7
Importing Data with R-base: read.table()	7
Tidyverse: readr package	8
Useful parameters for reading files	12
Properly formatting the column names	15
Reading excel data with readxl	16
Parsing Data	18
Parsing vectors	18
Kind of parsers	19
Parsing numbers	19
Parsing Strings	20
Parsing Factors	21
Parsing Dates and Times	21
Parsing a file	23
Writing Data	24
Improved writing	24
Other Files	25
Reading JSON data	25
Reading from APIs	26

Table of contents

API Example	27
Other cool APIs you can try!!	31
Processing High Volumes	32
Processing High Volumes of Data	32
Understand data volumes	32
Issue: data size is bigger than memory capacity	32
Block processing (chunking)	33
Block processing (chunking)	33
Three Options	33
Option 1: Local Sequential Processing	34
Option 2: Hadoop + MapReduce	34
Option 3: Apache Spark	35
When using each one?	35

Data Wrangling

Objective: Make data ready for their future **exploration** and **modelling**

We need to convert **raw data** into **processed data**

Raw Data

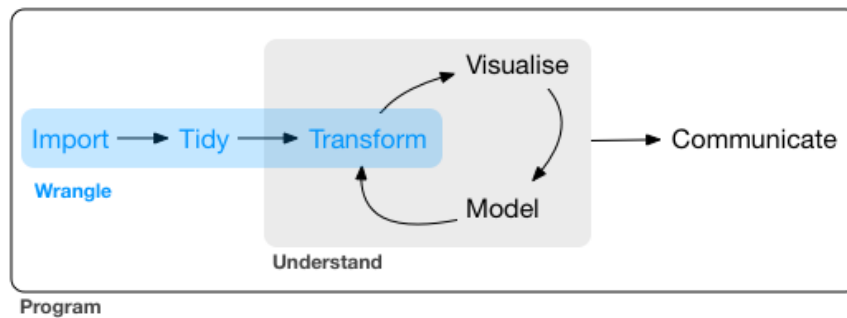
- Data as it appears on the source origin.
- Any manipulation has been done to them.

Processed Data

- Each variable in a column.
- Each observation in a row.
- Each observational unit is a cell.
- More complex data, interconnected tables.

The process work as follows...

1. Importing the data
2. Organizing the data



Basic Concepts

Understanding the path and working directory

When working in our environments R will need a working directory, the place where it is located inside your computer.

For managing it two functions are essential `getwd()` and `setwd()`

```
# Returns the current working directory  
getwd()
```

```
# Change the working directory  
setwd("./data")
```

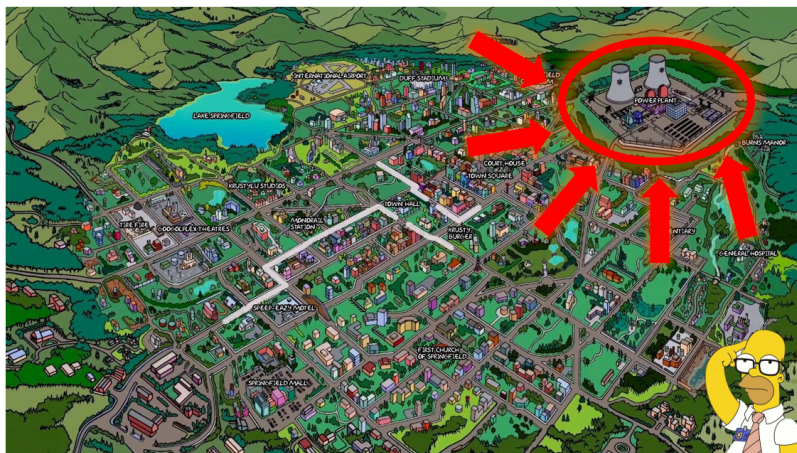
Creating a path

For navigating inside your computer you need to know how to build the paths... we can consider two types of paths when deciding how to tell our computer where we are...

- **Absolute Path**
- **Relative Path**

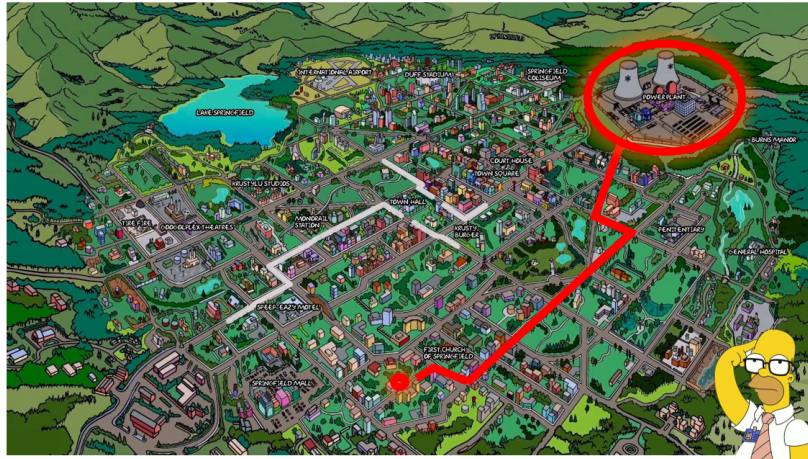
Let's understand the difference between each one with an example...

Homer works in the Nuclear Plant where it is? In this case we provide a full direction telling Homer where the place is.



This is what we call an **Absolute Path**, as we provide the exact position independent of the position.

Homer works in the Nuclear Plant how he can go? In this case we are trying to find the location considering an starting point and we need to provide a route to go from that point.



This is what we call a **Relative Path**, as we provide a route to the file considering the place we are in.

Now that we have the explanation of each kind of path definition, this is how we write each one in our code:

In the **absolute** one we define the position of the file starting from the *root folder* of our computer. Then we define each directory our computer needs to go to until finding the final destination.

```
# ABSOLUTE
# Linux / MacOS
setwd("/home/carviagu/data")

# Windows
setwd("C:/Users/carviagu/data")
```

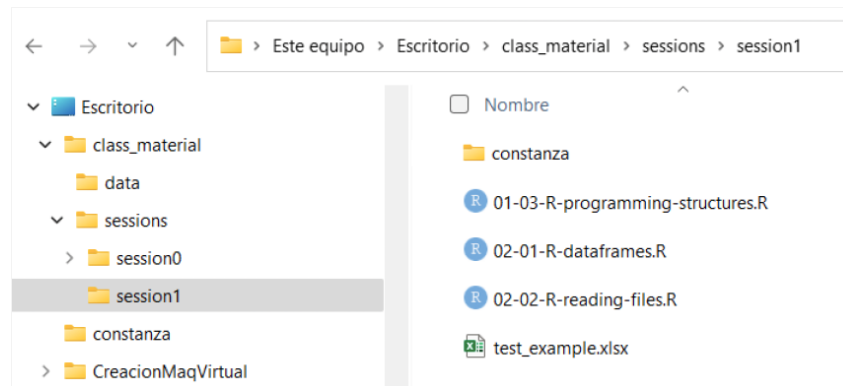
If you are not sure where your root folder is you can use "~. /" as a placeholder.

In the **relative** one we define the position of the file starting from the *current folder* we are in. Then we define each directory our computer needs to go to until finding the final destination.

Importing data

```
# RELATIVE
# from my current position
setwd("./data")
# from main folder
setwd("/Users/carviagu/data")
# one level up
setwd("../peter") # one level up
```

Here you have a practical example of relative paths. Considering the following folder structure...



As that you are at the session1 folder...

- You want to access test_example.xlsx file, your path could be "test_example.xlsx"
- You want to access a file called test2.csv at constanza folder, your path could be "constanza/test2.csv"
- You want to access a file called test3.csv located at sessions folder, your path could be "../test3.csv". As your going a level up in the folder structure we need to use ../ syntax.

Importing data

We will learn how to import, download and read data in R. Mainly focused on **tabular** data.

As we have seen we will continue using the **tidyverse** ecosystem, focused on the main packages available for importing data into our environments.

```
library(tidyverse)
```

We will follow this schema...

- Download data
- Read and Write data with tidyverse packages

Reading Data

Downloading data

We are going to learn how to create a folder called **data** from R and download data inside it.

1. Check if the folder **data** already exists in the current directory, if not you can create it. Use the following functions...

- `file.exists()`
- `dir.create()`

2. Download the data about elections participation on Madrid Community (Spain) from the [following url](#):

```
data_url <- "https://datos.comunidad.madrid/catalogo/dataset/929442fc-fef2-4f73-85a4-c4b02328531e"
download.file(data_url, destfile="../data/elections_data.csv")
list.files("../data")
```

CSV files

Comma Separated files are designed to export and manage data in plain text, they are more interesting for storing tabular data. They use a delimiter like `,` for example to separate each value. The first row will be column names.

Learn more about CSV file and their properties [here](#)

Importing Data with R-base: `read.table()`

In R base we have `read.table()` function to read data in simple way.

```
# This is how we can read data with read.table
# IMPORTANT: the file uses ; as separator
elections_data <- read.table("../data/elections_data.csv",
                             header = T, sep = ";")
head(elections_data, 5)
```

```
elections_data <- read.table("../data/elections_data.csv",
                             header = T, sep = ";")
head(elections_data, 5)
```

Tidyverse: readr package

	cod_muni	Municipio	Distrito	Seccion	Mesa	Censo	Certif_alta	Certif_correc.				
1	1	La Acebeda	1	1	U	64	0	0				
2	2	Ajalvir	1	1	A	859	0	0				
3	2	Ajalvir	1	1	B	863	0	0				
4	2	Ajalvir	1	2	A	766	0	0				
5	2	Ajalvir	1	2	B	871	0	0				
Electores		Votos_electores	Votos_Interventores	Votos_blancos		Votos_nulos						
1	64,0	54	0	0	1							
2	859,0	554	1	11	11							
3	863,0	543	1	6	3							
4	766,0	561	1	2	9							
5	871,0	624	2	14	5							
Votos_total		PP	PUM.J	PSOE	ULEG	PH	PCTE	MM.VQ	CS	PACMA	FE.DE.LAS.JONS	VOX
1	54	31	0	9	0	0	0	6	0	0	0	1
2	555	285	0	62	1	0	1	105	9	0	0	57
3	544	311	0	48	0	0	1	92	5	6	4	53
4	562	271	0	81	0	0	0	98	9	5	3	71
5	626	320	1	89	0	0	0	100	4	6	1	79
PODEMOS.IU.AV		PFE										
1	6	0										
2	13	0										
3	13	2										
4	13	0										
5	7	0										

Some important parameters...

- **header**: TRUE if the file contains the column names in its first row, FALSE otherwise.
 - **sep**: type of separator being use in the data: (,;|, etc.)
 - **na.strings**: choose which kind of character will represent the NA value.
 - **nrows**: maximum number of rows to read
 - **skip**: number of rows to skip from being read.
 - **col.names**: vector with the names to use as columns, if the file contains a first row with columns
- use header = F

Tidyverse: readr package

With **tidyverse** we have the **readr** package for reading data...

```
library(tidyverse) # library(readr) for using single package
```

Tidyverse: readr package

All readr function will do the data transformation and return a *tibble*. Some of the available functions...

- `read_csv()` - files using , as delimiter
- `read_csv2()` - files using ; as delimiter
- `read_tsv()` - files using tabulator as delimiter (|)
- `read_delim()` - files using another type of delimiter, we will need to provide it as an argument.

By default the function to read CSV files is `read_csv`.

```
iris_df <- read_csv("../data/iris.csv")
```

Rows: 150 Columns: 5

-- Column specification -----

Delimiter: ","

chr (1): Species

dbl (4): Sepal.Length, Sepal.Width, Petal.Length, Petal.Width

i Use ``spec()`` to retrieve the full column specification for this data.

i Specify the column types or set ``show_col_types = FALSE`` to quiet this message.

```
iris_df
```

A tibble: 150 x 5

	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
	<dbl>	<dbl>	<dbl>	<dbl>	<chr>
1	5.1	3.5	1.4	0.2	setosa
2	4.9	3	1.4	0.2	setosa
3	4.7	3.2	1.3	0.2	setosa
4	4.6	3.1	1.5	0.2	setosa
5	5	3.6	1.4	0.2	setosa
6	5.4	3.9	1.7	0.4	setosa
7	4.6	3.4	1.4	0.3	setosa
8	5	3.4	1.5	0.2	setosa
9	4.4	2.9	1.4	0.2	setosa
10	4.9	3.1	1.5	0.1	setosa

i 140 more rows

`read_csv` has been designed to read files that use a comma (,) as delimiter.

We can find some cases where the delimiter is different. The most common case occurs when we are working with files coming from *European* countries.

Look at the following file:

```
iris_df2 <- read_csv("../data/iris_esp.csv")
```

```
Rows: 150 Columns: 1
-- Column specification -----
Delimiter: ","
chr (1): Sepal.Length;Sepal.Width;Petal.Length;Petal.Width;Species

i Use `spec()` to retrieve the full column specification for this data.
i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
iris_df2
```

```
# A tibble: 150 x 1
  `Sepal.Length;Sepal.Width;Petal.Length;Petal.Width;Species`
  <chr>
1 5.1;3.5;1.4;0.2;setosa
2 4.9;3;1.4;0.2;setosa
3 4.7;3.2;1.3;0.2;setosa
4 4.6;3.1;1.5;0.2;setosa
5 5;3.6;1.4;0.2;setosa
6 5.4;3.9;1.7;0.4;setosa
7 4.6;3.4;1.4;0.3;setosa
8 5;3.4;1.5;0.2;setosa
9 4.4;2.9;1.4;0.2;setosa
10 4.9;3.1;1.5;0.1;setosa
# i 140 more rows
```

Looks something is not correct when reading the file, the whole data is stored in a single columns not separating them correctly. If we see the file we will detect that the variables are separated using ; delimiter, we can try another function then: read_csv2

```
iris_df2 <- read_csv2("../data/iris_esp.csv")
```

```
i Using ",", "." as decimal and "'.'" as grouping mark. Use `read_delim()` for more control.
```

```
Rows: 150 Columns: 5
-- Column specification -----
Delimiter: ";"
chr (2): Petal.Width, Species
num (3): Sepal.Length, Sepal.Width, Petal.Length
```

i Use ``spec()`` to retrieve the full column specification for this data.
 i Specify the column types or set ``show_col_types = FALSE`` to quiet this message.

```
iris_df2
```

```
# A tibble: 150 x 5
  Sepal.Length Sepal.Width Petal.Length Petal.Width Species
      <dbl>      <dbl>      <dbl> <chr>      <chr>
1         51         35         14 0.2      setosa
2         49         3         14 0.2      setosa
3         47         32         13 0.2      setosa
4         46         31         15 0.2      setosa
5          5         36         14 0.2      setosa
6         54         39         17 0.4      setosa
7         46         34         14 0.3      setosa
8          5         34         15 0.2      setosa
9         44         29         14 0.2      setosa
10        49         31         15 0.1      setosa
# i 140 more rows
```

Now the file has been read correctly. `read_csv2` has been designed to import data using a ; delimiter that is very frequent on *European* countries and *Hispanic* cultures. While other countries tend to use , separator by default.

In addition we can find files using more specific delimiters different from the frequent ones. For much more specific cases we can use `read_delim` that has been designed for reading custom delimiters.

```
iris_df3 <- read_delim("../data/iris.txt", delim = "|")
```

```
Rows: 150 Columns: 5
-- Column specification -----
Delimiter: "|"
chr (1): Species
dbl (4): Sepal.Length, Sepal.Width, Petal.Length, Petal.Width
```

i Use ``spec()`` to retrieve the full column specification for this data.
 i Specify the column types or set ``show_col_types = FALSE`` to quiet this message.

```
iris_df3
```

Useful parameters for reading files

```
# A tibble: 150 x 5
  Sepal.Length Sepal.Width Petal.Length Petal.Width Species
      <dbl>       <dbl>       <dbl>       <dbl> <chr>
1         5.1         3.5         1.4         0.2 setosa
2         4.9         3         1.4         0.2 setosa
3         4.7         3.2         1.3         0.2 setosa
4         4.6         3.1         1.5         0.2 setosa
5          5          3.6         1.4         0.2 setosa
6         5.4         3.9         1.7         0.4 setosa
7         4.6         3.4         1.4         0.3 setosa
8          5          3.4         1.5         0.2 setosa
9         4.4         2.9         1.4         0.2 setosa
10        4.9         3.1         1.5         0.1 setosa
# i 140 more rows
```

In this case we define on the function the delimiter being used in the file so R can separate each variable accordingly.

Useful parameters for reading files

In your reading functions you will find useful parameters you can define to adapt the reading depending on the file content.

Skip lines of the file

You can skip lines of files when reading them using skip argument, this argument will start reading the file ignoring the first number of lines given

```
# Use skip to avoid reading some lines
df <- read_csv("Metadata line information
               More information
               a,b,c
               1,2,3
               4,5,6", skip=2)
```

Warning: The `file` argument of `read\_csv()` should use `I()` for literal data as of readr 2.2.0.

```
# Bad (for example):
read_csv("x,y\n1,2")
```

```
# Good:
read_csv(I("x,y\n1,2"))
```

```
Rows: 2 Columns: 3
-- Column specification -----
Delimiter: ","
dbl (3): a, b, c
```

i Use ``spec()`` to retrieve the full column specification for this data.
 i Specify the column types or set ``show_col_types = FALSE`` to quiet this message.

```
df
```

```
# A tibble: 2 x 3
      a     b     c
  <dbl> <dbl> <dbl>
1     1     2     3
2     4     5     6
```

Manage column names

You can find files where column names are not given and the first line of the file is data you must have. By default first line will be always column names but you can override this behavior.

```
df <- read_csv("7,8,9
                1,2,3
                4,5,6", col_names=FALSE)
```

```
Rows: 3 Columns: 3
-- Column specification -----
Delimiter: ","
dbl (3): X1, X2, X3
```

i Use ``spec()`` to retrieve the full column specification for this data.
 i Specify the column types or set ``show_col_types = FALSE`` to quiet this message.

```
df
```

Useful parameters for reading files

```
# A tibble: 3 x 3
      X1    X2    X3
  <dbl> <dbl> <dbl>
1     7     8     9
2     1     2     3
3     4     5     6
```

Other option will be give specific column names, using the same parameter..

```
df <- read_csv("7,8,9
               1,2,3
               4,5,6", col_names=c("a", "b", "c"))
```

Rows: 3 Columns: 3

-- Column specification -----

Delimiter: ","

dbl (3): a, b, c

i Use `spec()` to retrieve the full column specification for this data.

i Specify the column types or set `show\_col\_types = FALSE` to quiet this message.

```
df
```

```
# A tibble: 3 x 3
      a     b     c
  <dbl> <dbl> <dbl>
1     7     8     9
2     1     2     3
3     4     5     6
```

Defining NA variables

Some data sets don't use NA or blank in their data and could use an specific value. In this case we are using "nan" when data is not available. We can tell the read function how to interpret a NA in the data...

```
df <- read_csv("a,b,c
               1,nan,3
               4,5,nan", na = "nan")
```

Properly formatting the column names

```
Rows: 2 Columns: 3
-- Column specification -----
Delimiter: ","
dbl (3): a, b, c

i Use `spec()` to retrieve the full column specification for this data.
i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

df

```
# A tibble: 2 x 3
      a     b     c
  <dbl> <dbl> <dbl>
1     1    NA     3
2     4     5    NA
```

Properly formatting the column names

Following conventions column names should follow some rules, we can make sure these rules are being applied with janitor package:

```
df <- read_csv("Name Surname, Is Smoker, PATIENT_AGE
                Juan Montero, YES, 35
                Maria Chacón, NO, 64") %>%
  janitor::clean_names()
```

```
Rows: 2 Columns: 3
-- Column specification -----
Delimiter: ","
chr (2): Name Surname, Is Smoker
dbl (1): PATIENT_AGE
```

```
i Use `spec()` to retrieve the full column specification for this data.
i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

df

```
# A tibble: 2 x 3
  name_surname is_smoker patient_age
  <chr>        <chr>        <dbl>
1 Juan Montero YES          35
2 Maria Chacón NO           64
```

Reading excel data with readxl

One of the most common file formats you may find are excel files, for reading them there is a special library not found in readr, you need to work with readxl

```
library(readxl)
```

You can read an excel file using read_excel function...

```
iris_data <- read_excel("../data/iris.xlsx", sheet = 1)
head(iris_data)
```

```
# A tibble: 6 x 5
  Sepal.Length Sepal.Width Petal.Length Petal.Width Species
      <dbl>         <dbl>         <dbl>         <dbl> <chr>
1         5.1         3.5         1.4         0.2 setosa
2         4.9         3         1.4         0.2 setosa
3         4.7         3.2         1.3         0.2 setosa
4         4.6         3.1         1.5         0.2 setosa
5          5         3.6         1.4         0.2 setosa
6         5.4         3.9         1.7         0.4 setosa
```

Consider that we have provided the sheet number as an argument so the function knows which sheet to read. By default it will always read the first sheet (independently if it is hidden or not).

You can also provide the name of sheet instead of the number.

```
iris_data <- read_excel("../data/iris.xlsx",
  sheet = "Sheet1")
head(iris_data)
```

```
# A tibble: 6 x 5
  Sepal.Length Sepal.Width Petal.Length Petal.Width Species
      <dbl>         <dbl>         <dbl>         <dbl> <chr>
1         5.1         3.5         1.4         0.2 setosa
2         4.9         3         1.4         0.2 setosa
3         4.7         3.2         1.3         0.2 setosa
4         4.6         3.1         1.5         0.2 setosa
5          5         3.6         1.4         0.2 setosa
6         5.4         3.9         1.7         0.4 setosa
```

Reading excel data with readxl

For excel you can work with the ranges available on the sheets, so you can be more specific regarding which part to read.

```
iris_data <- read_excel("../data/iris.xlsx",
  sheet = 1, range = "C1:E40")
head(iris_data)
```

```
# A tibble: 6 x 3
  Petal.Length Petal.Width Species
      <dbl>      <dbl> <chr>
1         1.4         0.2 setosa
2         1.4         0.2 setosa
3         1.3         0.2 setosa
4         1.5         0.2 setosa
5         1.4         0.2 setosa
6         1.7         0.4 setosa
```

You can use `cell_cols()` or `cell_rows()` with range argument to specify only one part of the range.

Parsing Data

read_csv function realizes a parsing of text file. This means it converts a text file in an organized data structure.

Parsing vectors

For understanding better what it does that we need to understand **parsing** functions. These functions will transform a vector of characters into an specialized data vector.

```
str(parse_logical(c("T", "F", "F")))
```

```
logi [1:3] TRUE FALSE FALSE
```

```
str(parse_integer(c("1", "4")))
```

```
int [1:2] 1 4
```

```
str(parse_date(c("2022-01-04")))
```

```
Date[1:1], format: "2022-01-04"
```

All parse functions are uniform: the first argument is the vector to parse and the na argument lets us decide with value treat as a missing value.

```
parse_integer(c("1", "4", "."), na=".")
```

```
[1] 1 4 NA
```

If something goes wrong during the process a warning message will be displayed to inform us

```
parse_integer(c("1", "4", "c"))
```

```
Warning: 1 parsing failure.
```

```
row col      expected actual
  3  -- no trailing characters    c
```

Kind of parsers

```
[1] 1 4 NA
attr(,"problems")
# A tibble: 1 x 4
  row col expected actual
<int> <int> <chr>      <chr>
1     3    NA no trailing characters c
```

Kind of parsers

- `parse_logical()` and `parse_integer()` for logical and integer values.
- `parse_double()` and `parse_number()` for numbers (second one more flexible)
- `parse_factor()` for categorical values (factors)
- `parse_datetime()`, `parse_date()` and `parse_time()` for times and dates.

Parsing numbers

Numbers are quite complex as they have three main problems:

- 1) Numbers can be described differently depending on the culture/language: 1,46 or 1.45

```
parse_double("1,45", locale = locale(decimal_mark = ","))
```

```
[1] 1.45
```

- 2) Sometimes we can find symbols in the numbers: 100%

```
parse_number("100$")
```

```
[1] 100
```

```
parse_number("39%")
```

```
[1] 39
```

```
parse_number("Me costó 100 €")
```

```
[1] 100
```

Parsing Strings

- 3) Related with the first issue, representation can change on aggregation level: 1.000.000 or 1000000 or 1,000,000 or 1000K or 1M

```
parse_number("%100,330")
```

```
[1] 100330
```

```
parse_number("100.330", locale = locale(grouping_mark = "."))
```

```
[1] 100330
```

Parsing Strings

Computers represent text using hexadecimal code (look for [ASCII](#))

```
charToRaw("Hello")
```

```
[1] 48 65 6c 6c 6f
```

The mapping between hexadecimal values and characters are called **encoding**. By default **readr** uses UTF-8, general by default.

Sometimes in can fail with old systems files:

```
x1 <- "Nos vemos ma\xflana"
parse_character(x1, locale = locale(encoding = "ISO-8859-1"))
```

```
[1] "Nos vemos mañana"
```

We can avoid issues, analyzing the text prior to find the best encoding

```
x1 <- "Nos vemos ma\xflana"
guess_encoding(charToRaw(x1))
```

```
# A tibble: 3 x 2
  encoding confidence
  <chr>         <dbl>
1 ISO-8859-1     0.52
2 ISO-8859-2     0.35
3 ISO-8859-9     0.35
```

Parsing Factors

In this case we will need to provide `parse_factor()` a vector with the possible levels:

```
parse_factor(
  c("man", "woman", "woman", "unicorn"),
  levels = c("man", "woman")
)
```

Warning: 1 parsing failure.

```
row col      expected actual
  4  -- value in level set unicorn
```

```
[1] man  woman woman <NA>
attr("problems")
# A tibble: 1 x 4
   row  col expected      actual
  <int> <int> <chr>      <chr>
1     4    NA value in level set unicorn
Levels: man woman
```

Parsing Dates and Times

`parse_datetime()` for dates with hour. In the following order: *year, month, day, hour, minute and second*

```
parse_datetime("2024-01-01T1015")
```

```
[1] "2024-01-01 10:15:00 UTC"
```

`parse_date()` for dates. In the following order: *year, month, day* using - or / as separator

```
parse_date("2024/01/01")
```

```
[1] "2024-01-01"
```

`parse_time()` for time. In the following order: *hour, minute, seconds* using : as separator. am or pm are optional.

```
parse_time("10:15 am")
```

10:15:00

You can define a format of the input to help the parsing...

```
parse_date("01/04/2023", "%d/%m/%Y")
```

Warning: 1 parsing failure.

row	col	expected	actual
1	--	date like %d/%m/%y	01/04/2023

[1] NA

Year	Month	Day
• %Y 4 digits • %y 2 digits*	• %m 2 digits • %b "jan" • %B "january"	• %d 2 digits • %e space

*00-69 -> 2000-2069, 70-99 -> 1970-1999

You can define a format of the input to help the parsing...

```
parse_time("10-30", "%H-%M")
```

10:30:00

Time	—
• %H 0-23 hours (24) • %p 0-12 hours (12), use with %p • %p AM/PM • %M minutes	• %s seconds (integer) • %OS real seconds • %Z time zone (e.g. America/Chicago) • %z UTC offset (e.g. +0800)

Discover more about formatting dates [here](#)

Parsing a file

How readr parse correctly the data?

readr will guess the correct the type of each column as follows:

- Read the first 1000 rows
- Guess the type using a determined heuristic

```
guess_parser("2022-01-01")
```

```
[1] "date"
```

```
str(parse_guess("2022-01-01"))
```

```
Date[1:1], format: "2022-01-01"
```

Sometimes it can fail...

- It is possible that the first 1000 rows have special cases (e.g. 1000 integer values in a double variable)
- Missing values can appear at the column.

It is suggested to try to inform the type of each column when it is possible.

```
# Not specifying the types  
read_csv("../data/df-na.csv")
```

We can force it two ways...

```
# Specifying the types  
read_csv("../data/df-na.csv",  
          col_types = list(col_double(),  
                           col_character(),  
                           col_character()))
```

```
# Other alternative  
read_csv("../data/df-na.csv", col_types="ncc")
```

Writing Data

The readr library include functions to write dat and store it in files outside R.

Three main functions:

- `write_csv()` to write csv files
- `write_tsv()` to write tsv files
- `write_delim()` to write with different delimiters

Using them:

```
write_csv(df, "df.csv")
```

Improved writing

Last writing functions are good, but don't save the column typification making necessary specifying each time when reading. But we can use `write_rds()`, `read_rds()` in readr.

This functions will keep the data in binary RDS format keeping metadata for helping reading it properly.

```
write_rds(iris, "../data/iris.RDS")  
iris_tibble <- read_rds("../data/iris.RDS")  
iris_tibble
```

Other Files

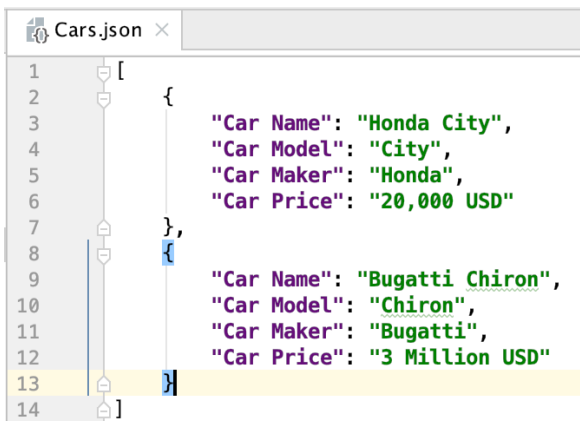
There are plenty of libraries useful for reading/writing from different kind of sources in R:

- **haven** for SPSS, Stata, SAS
- **readxl** and **writexl** for reading and writing excel files
- **DBI** together with **RMySQL** or **RSQLite** for launching SQL queries to databases.
- **jsonlite** for JSON

Reading JSON data

What is JSON?

- JSON are the letters of Javascript Object Notation, is a lighter storing data format.
- It is very frequently used for APIs, data is stored using objects pair key-value.



For working with JSON in R we need to use jsonlite library.

```
library(jsonlite)

# Reading JSON
iris_df <- read_json("../data/iris.json", simplifyVector = TRUE)

head(iris_df)
```

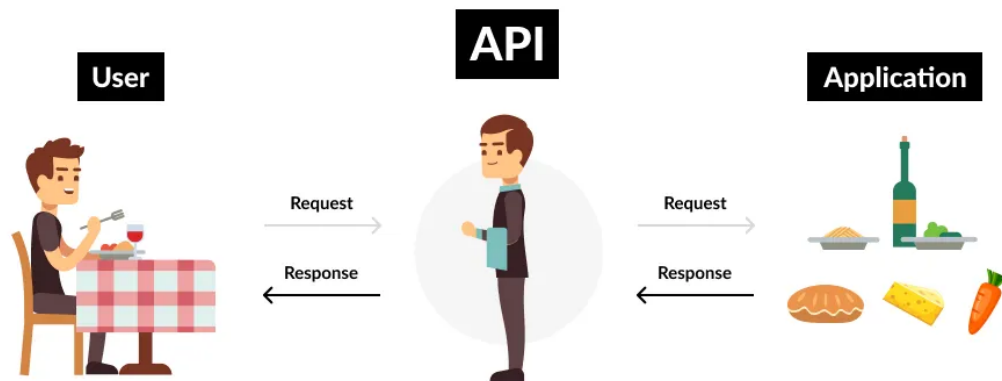
Reading from APIs

What is an API

An application programming interface (API) is a set of defined rules that explain how computers or applications communicate with one another. In this section will learn how to use these tools for requesting data from a web server (get method).

They can also be used for submitting forms (post method) or any interaction that the user can be allowed to —delete, put, etc.

How it works



1. User or computer asks to the API following a pre-defined syntax. Normally this petitions are done with an URL following a defined structure.
2. The API receives and verifies the petition and translates this petition to the corresponding system or database to retrieve and send the answer to the user.
3. User receives the answer.

This way the system or database is protected and the access is controlled via the API, that verifies and manages petitions.

Using APIs with R: httr2

In R we are going to use tidyverse library called httr2 that will help us using APIs.

```
library(httr2)
```



API Example

We are going to work with the Instituto Nacional de Estadística (INE) API for retrieving information.

The portal has an API where we can obtain data [here](#).

A normal URL for making request to the API has the following format...

`https://servicios.ine.es/wstempus/js/{idioma}/{función}/{input}[?parámetros]`

Obtaining data from PCI-IPC

We want to obtain information regarding Consumer Prices Index (PCI-IPC) evolution in Spain for different groups.

For these we will need to manage how to implement the URL and make the request. You can use in these case an URL generator [here](#)

```
# Create basic request (by default is a GET, we can change it)
req <- request("https://servicios.ine.es/wstempus/js/ES/DATOS_TABLA/50903?date=20240101:20240630")
req
```

<httr2_request>

GET https://servicios.ine.es/wstempus/js/ES/DATOS_TABLA/50903?date=20240101:20240630

Body: empty

Performing requests

Once we have configured the request, we can perform it...

```
resp <- req_perform(req)
resp
```

```
<httr2_response>
GET https://servicios.ine.es/wstempus/js/ES/DATOS_TABLA/50903?date=20240101:20240630
Status: 200 OK
Content-Type: application/json
Body: In memory (126715 bytes)
```

Responses Components: Status

If all goes correct we will receive a 200 response code (OK)

```
resp_status(resp)
```

```
[1] 200
```

```
resp_status_desc(resp)
```

```
[1] "OK"
```

But something can go wrong... in that cases...

- 404 - File not found
- 403 - Permission denied.
- 500 - Generic failure.

Responses Components: Headers

```
resp_headers(resp)
```

```
<http2_headers>
```

```
Date: Sun, 30 Aug 2026 08:22:45 GMT
```

```
Server: Apache
```

```
Last-Modified: Sun, 30 Aug 2026 07:49:22 GMT
```

```
X-OneAgent-JS-Injection: true
```

```
Server-Timing: dtRpid;desc="-1893362553"
```

```
Access-Control-Allow-Origin: *
```

```
Access-Control-Allow-Headers: Content-Type
```

```
X-Frame-options: DENY
```

```
Cache-Control: public
```

```
Expires: Thu, 1 Jan 1970 00:00:00 GMT
```

```
ETag: "1788076163:dtagent103432607151238112T1V"
```

```
Content-Language: es
```

```
Age: 2005
```

```
Content-Length: 126715
```

```
Content-Type: application/json; charset=UTF-8
```

```
Set-Cookie: dtCookiec132n50s=v_4_srv_7_sn_F3E7175B52E0E84810F767DEA3503753_perc_100000_ol_0_mul_13A50b3efb0f7e34fcb_0; Path=/; Domain=.ine.es
```

```
Set-Cookie: INE_WEB_COOKIE=es;Max-Age=31536000;Path=/
```

```
Set-Cookie: BIGipServerwstempus.ine.es_pool=2528197386.20480.0000; path=/; Httponly; Secure
```

```
Pragma: public
```

Responses Components: Body

Your requested data will be in the response body, we will obtain this information using JSON format...

```
body_resp <- resp_body_json(resp)
```

```
result_data <- body_resp[[1]]$Data
```

```
result_data[[1]]
```

```
$Fecha
```

```
[1] 1.717193e+12
```

```
$FK_TipoDato
```

```
[1] 1
```

```
$FK_Periodo  
[1] 6
```

```
$Anyo  
[1] 2024
```

```
$Valor  
[1] 116.212
```

```
$Secreto  
[1] FALSE
```

Obtaining the data

We can extract and prepare the data...

```
ipc_data <- as_tibble(  
  list(  
    "year" = c(2024),  
    "month" = c(0),  
    "value" = c(0)  
  )  
) %>% filter(month != 0)  
  
for (res in result_data){  
  temp_data <- as_tibble(  
    list(  
      "year" = c(res$Anyo[1]),  
      "month" = c(res$FK_Period[1]),  
      "value" = c(res$Valor[1])  
    )  
  )  
  ipc_data <- rbind(ipc_data, temp_data)  
}  
  
ipc_data
```

```
# A tibble: 6 x 3  
  year month value  
<int> <int> <dbl>
```

Other cool APIs you can try!!

1	2024	6	116.
2	2024	5	116.
3	2024	4	115.
4	2024	3	115.
5	2024	2	114.
6	2024	1	113.

Other cool APIs you can try!!

- <https://medium.com/@diego.coder/las-mejores-apis-públicas-para-practicar-programación-a7419ecc968e>
- <https://www.thecocktaildb.com>
- Transport of London API. You can read and try the TFL_Api_Example.R file and see how it works.

Processing High Volumes

In the course we are always working with small size files, easy to manage for learning purposes. In reality, big amounts of data are processes and some knowledge on how to manage these big files is important.

Our computers have some limitations that may interfere with our processes.

Processing High Volumes of Data

We have work with files of **less than 1 GB of RAM** use with the tidyverse in class.

In reality you will work with **hundreds of GB or TB**. Here we have two issues:

- **Memory limit:** The data size is higher than the RAM capacity of your computer.
- **Read or Processing limitations:** Maybe the file can be stored and processed, but due to capacity limitations all processing is time consuming.

Understand data volumes

Here you can find a summary of sizes in computer storage...

Unit	Name	Value
K	Kilo	1024 B (bytes)
M	Mega	1024 KB (kilobytes)
G	Giga	1024 MB (megabytes)
T	Tera	1024 GB (gigabytes)
P	Peta	1024 TB (terabytes)

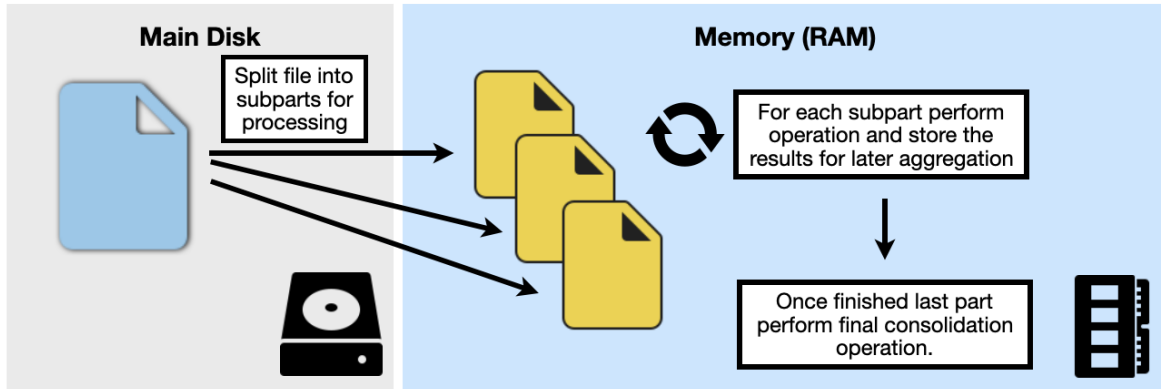
Issue: data size is bigger than memory capacity

Resource	Normal Capacity	Estimated Speed (Read/Write)
RAM	8–64 GB	10–100 GB/s
SSD	0.5–4 TB	0.5–3 GB/s
HDD	4–20 TB	100–250 MB/s

Accessing to a memory disk can be slow, very frequent with HDD disks. Efficient operations should minize read and write operations to the disk.

Block processing (chunking)

The main strategy when managing big files is processing them in different pieces (blocks or chunks of information).



Block processing (chunking)

General rule (Ex: compute the average of a big list of numbers):

1. **Initialize:** Create accumulators in RAM (sum = 0, n = 0).
2. **Iterate (loop):**
 - Read a chunk of data from the disk..
 - Process that chunk in RAM (compute sum_chunk, n_chunk).
 - Update (sum += sum_chunk, n += n_chunk).
3. **Finalize:** When the whole file has been processed, compute the desired metric (average = sum / n).

This principle applies to both a single computer and a distributed environment (multiple computers)

Three Options

Option	Computation	Source	Speed
Local Partition	1 computer	Local Disk	Medium
Hadoop + MapReduce	Multiple computers	Distributed Disk	Low (depending on disk)

Option 1: Local Sequential Processing

Option	Computation	Source	Speed
Spark	Multiple computers	Distributed RAM (disk if is not enough)	High (interactive)

Option 1: Local Sequential Processing

- A single execution thread will read sequentially all file blocks from the main disk and process them.
- Limitation: The performance is limited by the I/E speed from disk and CPU capacity.

```
con <- file("numbers.csv")
chunk <- 1e6
sum <- 0; n <- 0
repeat {
  x <- scan(con, n = chunk, quiet = TRUE) # read from main disk
  if (length(x) == 0) break
  sum <- sum + sum(x)
  n <- n + length(x)
}
close(con)
average <- sum / n
```

Limitations: sequential; any issue will need a reboot of the process; no computer paralelism.

Option 2: Hadoop + MapReduce

- **HDFS:** split the file in blocks and keep them in **different computers** with **copies**.
- **MapReduce:** **parallel** processing and **combination** of results.

Global Average (flow):

```
Nodo A → (sumA, nA)
Nodo B → (sumB, nB)
...
Combine → average = (Σ sum_i) / (Σ n_i)
```

- Computation done in the **RAM** of each node (computer).
- **Intermediate** results are red/written on the **HDFS (disk)**.
- **Solid**, designed for **lot processing** (not interactive).

Option 3: Apache Spark

- Splits the information in nodes too, but it maintains **data and intermediaten results in RAM** when possible.
- Tries to not access the disk between steps.
- Useful for **interactive analysis** and **repetitive algorithms**.

Spark using R (minimum):

```
library(sparklyr); library(dplyr)
sc <- spark_connect(master = "local")      # or cluster
copy_to(sc, mtcars, overwrite = TRUE) %>%
  summarise(media_hp = mean(hp)) %>% collect()
```

When using each one?

Local Partition

- File **less than RAM** capacity or you only have **one computer**.
- Simple operations (sums, counting, average).

Hadoop + MapReduce

- **Very big** file (multi-TB or more) and **no** need of quick interaction.
- **Scheduled** proceses (daily/weekly), high **reliability**.

Spark

- **Big** files (10 GB – tens of TB) with need of **quick response**.
- **Interactive analysis**, repetitive tests.
- Reuse **R/dplyr** (sparklyr) with the same code style.